KellyBench: Can Language Models Beat the Market?

Thomas Grady[1], **Kip Parker**[1], **Iliyan Zarov**[1], **Henry Course**[1], **Chengxi Taylor**[1] **and Ross Taylor**[1†]
[1]General Reasoning, Inc., [†]Correspondence to hello@gr.inc

Language models are saturating benchmarks for procedural tasks with narrow objectives. But they are increasingly being deployed in long-horizon, non-stationary environments with open-ended goals. In this paper we introduce `KellyBench`, an environment for evaluating sequential decision-making in sports betting markets. Agents are placed in a sequential simulation of the 2023–24 English Premier League season and tasked with maximising their long-term bankroll growth. They are given detailed historical data, including advanced statistics, lineups, and public odds. To succeed they must build machine learning models, identify edge in public markets, and adapt as the environment changes over time. We find that all frontier models evaluated lose money over the course of the season, with many experiencing ruin. To judge strategy sophistication, we use a human expert rubric to grade each model and find their approaches to be unsophisticated compared to human baselines. We believe our results highlight a need for a cultural shift in evaluations away from fixed task sets towards complex worlds that test long-horizon, sequential decision-making. `KellyBench` is available as an open-access API endpoint on OpenReward at `https://openreward.ai/GeneralReasoning/KellyBench`.

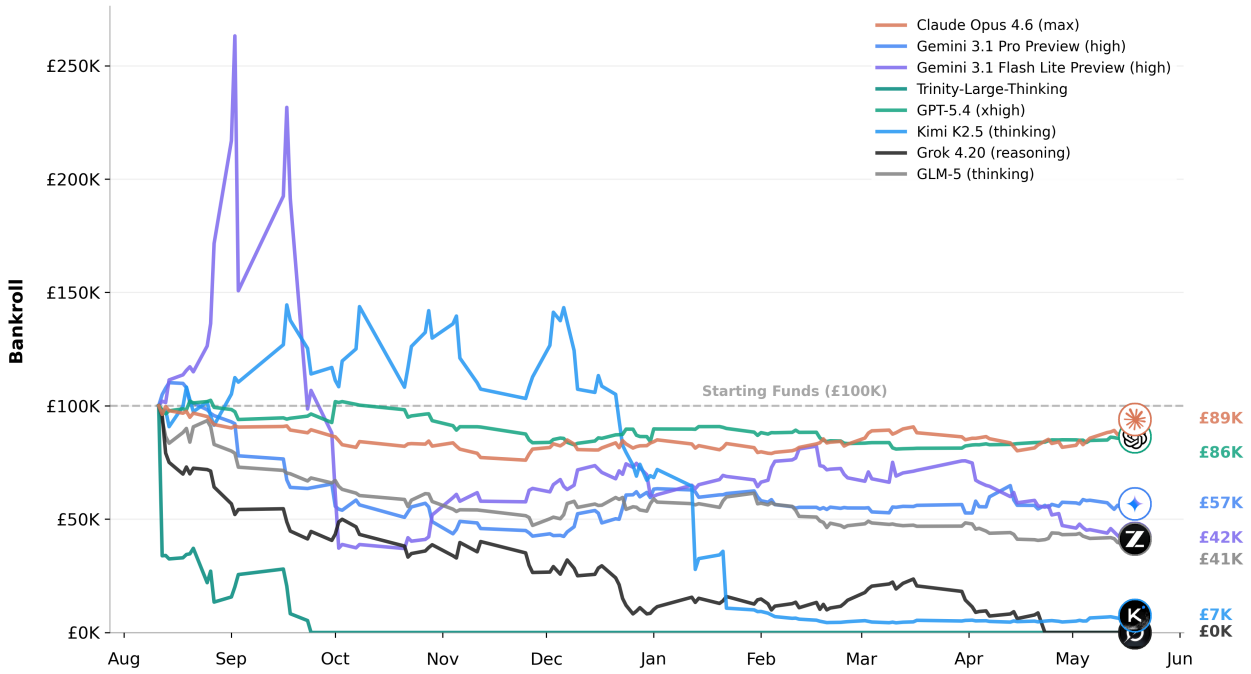


Figure 1 | **Model Performance on KellyBench**. KellyBench tasks models with developing machine learning betting strategies for the 2023/24 English Premier League season with the goal of maximising long-term bankroll growth. No model makes a return on average across 3 seeds. Models also fail to adapt strategies in response to failure. Initial bankroll is normalised to £100K for display purposes.

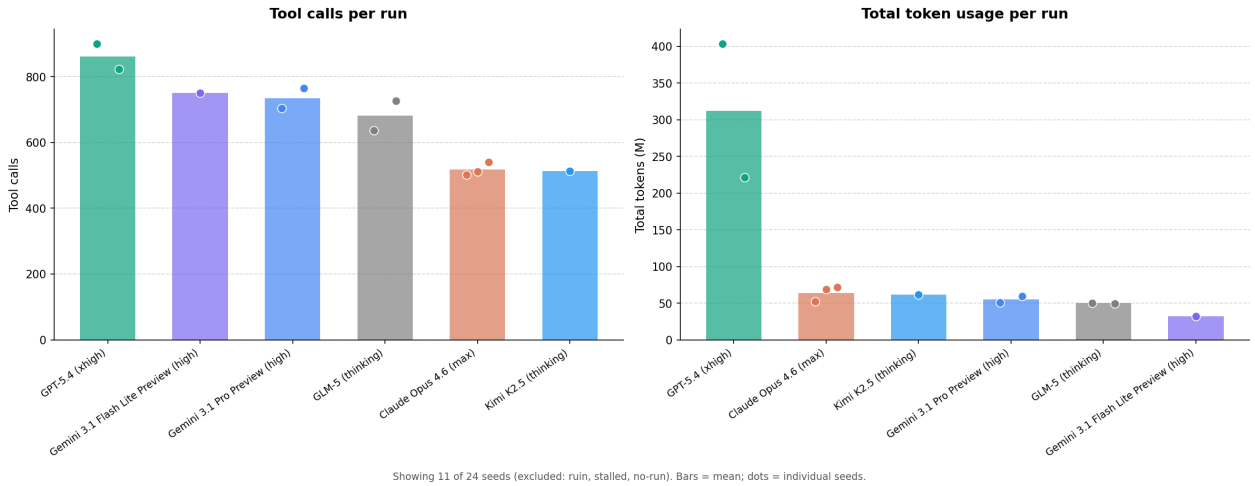


Figure 2 | **Long-Horizon Sequential Decision Making**. Agents we evaluated in `KellyBench` used 500-900 tool calls to complete an episode and 32-450 million tokens. Runs on `KellyBench` are expensive: one of the GPT-5.4 (xhigh) seeds cost $2,012 to complete a single episode.

1. Introduction

One definition of intelligence is inductive inference, the ability to learn from experience, build models from acquired knowledge, and use those models for prediction (Hutter, 2007). However, in the real world agents must convert predictions into useful actions. Betting markets provide an example of this difference. A predictive model that is more accurate than the public on average need not be profitable, and can lead to ruin if the edge on some bets is wrongly estimated (Benter, 2008; Thorp, 1997).

Many popular evaluations of language models do not measure intelligence in this sense of learning from experience. Instead, they typically consider stationary environments, well-specified tasks, and sparse end-of-episode feedback. For example, one task in the popular `terminalbench2` evaluation asks an agent to "implement an adaptive-rejection sampler as described in Gilks et al. (1992)". While this tests procedural competence, it does not test the ability to formulate and revise models in light of experience (Hughes et al., 2024; Merrill et al., 2026).

The real world is also non-stationary where the underlying "rules" change over time. However, most existing benchmarks have fixed behaviours. For example, the knight and the bishop have exactly the same behaviour in any game of chess, but a financial security will change behaviour under a new market regime, and an athlete's ability will change after a long-term injury (Ang and Timmermann, 2012; Johns et al., 2021). In European football, for example, home advantage was found to decline in strength at the end of the 2019-2020 season under pandemic crowd restrictions, creating a bias in older predictive models (Hill and Van Yperen, 2021). This suggests a potential capability gap between models acting in static environments at training-time and dynamic ones at test-time.

To study these issues, we introduce `KellyBench`. `KellyBench` is an open-ended, non-stationary environment for measuring the ability of language models to make money in sports betting markets. `KellyBench` uses real market odds from the 2023/2024 English Premier League season and asks agents to bet from a bankroll for each matchday. Agents are given extensive historical data, including advanced statistics, lineups, and past market odds. They must develop machine learning models, identify edge relative to the market, and manage risk so as to maximise long-run bankroll growth.

Every model we evaluate on `KellyBench` loses money over the course of the season and several models experience ruin. The best performing model Claude Opus 4.6 achieves an average return on

investment of −11% over the course of the season. Only 2/24 model seeds finish the simulation with a positive return, and both of those models experienced ruin in other seeds. Via qualitative analysis of their trajectories, we find frontier models have poor adaptivity and low competence in accounting for potential estimation error and non-stationarity. In other words, the current generation of frontier models cannot consistently beat the market in our evaluation setup.

We also introduce a novel process-based measure of competence called *sophistication*. Backtests can be subject to variance, so we consult experts with experience at quantitative betting funds to construct a 44-point rubric judging strategy sophistication. Using these rubrics, model strategies are consistently judged as unsophisticated relative to human baselines. The best performing model Claude Opus 4.6 has a sophistication score of 32.6%. Therefore, even with the limitations of our benchmark setup, and possibly high market efficiency, we believe there is considerable room for models to improve.

To examine whether our headline results are due to under-elicitation, we perform additional ablations with Claude Opus 4.6. We give Opus 4.6 access to relevant historical literature that human experts regard as seminal for constructing high-performing models, and also evaluate it with a native Claude Code harness. In these ablations, neither intervention improves results and the model continues to lose money on average over three seeds.

We conclude by discussing several limitations, including the possibility of underelicitation from the use of a single-agent harness and the possibility that the data provided to the agent are insufficient to outperform the market. However, in spite of these limitations, our conclusion remains that models are systematically underperforming humans, and that much better performance is possible with the tools and data they are given. We argue a cultural shift is needed in making evaluations, moving away from fixed tasksets towards complex worlds where agents must learn from experience under uncertainty.

2. Background

2.1. Quantitative Betting Approaches

Probability theory arose in part from practical questions about gambling. For example, Cardano's sixteenth-century *Liber de ludo aleae* combines advice on games of chance with reflections on probability and calculation (Bellhouse, 2005). Related ideas reappear in twentieth-century subjectivism, especially in Ramsey and de Finetti, where degrees of belief are interpreted through betting behaviour. In particular, Dutch book arguments are used to motivate Bayesian probability by excluding systems of belief that admit sure loss or "money pumps" (de Finetti, 1937; Ramsey, 1931).

The mathematical foundations of gambling were later reformulated in the mid-twentieth century through information theory. Shannon's theory of communication provided a quantitative language to measure informational advantage (Shannon, 1948). Kelly translated this into a repeated betting problem by showing that, when an agent has access to side information, the strategy that maximises expected log-wealth also maximises the long-run exponential growth rate of capital (Kelly, 1956).

In particular, let X denote the outcome of a repeated gamble, Y the side information observed before betting, $o(x)$ the gross odds on outcome x, and $b(x \mid y)$ the fraction of wealth staked on x after observing $Y = y$. The agent seeks to maximise the expected logarithmic growth rate:

$$W(b) = \sum_{x,y} p(x, y) \log\big(b(x \mid y)o(x)\big).$$

In the canonical model, the optimal strategy is to bet in proportion to the conditional probabilities $b^*(x \mid y) = p(x \mid y)$. Under fair odds this gives:

$$W^* = H(X) - H(X \mid Y) = I(X;Y),$$

so that the value of side information is exactly the mutual information between Y and X. Informational advantage is thus converted directly into long-run capital growth.

The same point may be expressed in divergence form. If market odds imply a distribution q, the true distribution is p^*, and the agent sizes bets according to beliefs p, then the expected log-growth is:

$$g(p; p^*, q) = \sum_x p^*(x) \log \frac{p(x)}{q(x)} = D_{\mathrm{KL}}(p^* \| q) - D_{\mathrm{KL}}(p^* \| p).$$

Growth is positive when the agent's model is closer to the truth than the market-implied distribution. Superior calibration becomes an informational edge which can be converted into compounding gains.

In practice, the Kelly criterion makes this link operational. For a binary gamble with win probability p and net odds r, it prescribes staking a fraction of wealth:

$$f^* = \frac{rp - (1-p)}{r}.$$

These principles were first operationalised at scale by Ed Thorp, who treated gambling and finance as structurally similar problems of edge detection, stake sizing, and risk control. In *Beat the Market* and later at Princeton/Newport, this framework was applied to hedged mispricings in warrants, convertibles and equities (Thorp and Kassouf, 1967). The same capital-growth logic that governed bet sizing in blackjack and sports betting was used to allocate capital across trading opportunities in securities markets (Thorp, 2005, 2008).

In betting markets specifically, Bill Benter and Alan Woods supplied a more direct large-scale operationalisation of Kelly-style staking. Benter's account of Hong Kong racing describes a computer-based handicapping system built from fundamental variables and a logit model, which was then refined by combining the model's probabilities with the public's implied probabilities in order to reduce bias and identify genuine overlays (Benter, 2008). Wagering was treated as a separate optimisation problem. Benter employed Kelly staking, but with two important modifications: fractional Kelly to reduce sensitivity to estimation error, and limits arising from the bettor's own market impact.

These examples suggest what is required of highly capable AI systems in betting markets. The task is not merely to construct an accurate fundamental model of the world, but to identify *relative* edge against the market, and to distinguish genuine opportunity from estimation error. In this sense, the task extends beyond inductive inference to sequential decision-making under uncertainty.

2.2. Machine Learning Models for Football Betting

Statistical models play a large role in feature design due to the presence of non-stationarity. Team and player strengths evolve over time, and this has historically been handled more naturally by approaches such as statistical state-space models rather than modern machine learning methods (Harvey, 1989).

For football prediction, Dixon and Coles (1997) introduce a dynamic team-ability model based on exponentially weighted Poisson likelihoods. Glickman and Stern (1998) develop a Bayesian state-space model for NFL scores, while Crowder et al. (2002) model attacking and defensive strengths as latent processes in a non-normal state-space framework. Similarly, Koopman and Lit (2015) propose a dynamic Poisson model with stochastically evolving intensities and show that its forecasts can support

profitable betting strategies. State space models can be computationally expensive, so Taylor (2016) proposes a score-driven alternative that avoids the need for large-scale MCMC methods[1].

Other features may be obtained through machine learning methods for constructing advanced statistics. An important example in football is the expected goals (xG) model, which estimates shot-conversion probabilities from event or tracking data. Anzer and Bauer (2021) show that a strong xG model can be built using extreme gradient boosting on synchronised positional and event data, while later work by Mead et al. (2023) and Bandara et al. (2024) improves prediction by adding features about the sequence of actions preceding each shot.

These dynamic-ability and advanced-statistics features can in turn be combined with a broader collection of fundamental features, including head-to-head records, weather, travel distance, stadium altitude, and many other facets of the world (Borghesi, 2007; Coleman, 2017; Nichols, 2014; Van Damme and Baert, 2019; Watanabe et al., 2017). The football betting problem is therefore open-ended: there is a large space of possible features to hypothesise and test.

2.3. Related Work

Machine Learning Engineering Evaluations. Existing work has mostly considered narrow evaluations of procedural ability in fitting and evaluating models. MLE-Bench evaluates models on 75 narrowly scoped offline Kaggle competitions (Chan et al., 2025). MLGym constructs 13 open-ended AI research tasks spanning areas such as supervised learning, reinforcement learning, and game theory (Nathani et al., 2025). More recently, PostTrainBench focuses on post-training language models, asking agents to improve a base language model on a target benchmark under bounded compute (Rank et al., 2026). KellyBench differs from these benchmarks in that it is non-stationary, has a larger potential feature space, and challenges agents to operationalise models in a sequential decision-making environment.

Forecasting Evaluations. Recent work has produced a growing list of forecasting benchmarks for AI models. ForecastBench introduces a dynamic benchmark of unresolved future-event questions (Karger et al., 2025). FutureX provides a large live benchmark for predicting the future with daily updates (Zeng et al., 2025). Prophet Arena evaluates models on continuously collected live forecasting tasks with accuracy, calibration and economic-value metrics (Yang et al., 2026). Bench to the Future offers a pastcasting environment for repeatable evaluation of forecasting agents (Wildman et al., 2025). Lastly, OpenForecaster evaluates forecasting performance on a held-out OpenForesight test set and on FutureX, reporting accuracy and calibration-oriented gains while using an offline news corpus to reduce temporal leakage during evaluation (Chandak et al., 2025).

Existing forecasting benchmarks provide valuable tests of inductive inference, but they evaluate agents on sparse, heterogeneous, and one-off event questions. Because sports betting has many structurally similar events, `KellyBench` provides more opportunities to test if a model can consistently translate estimated edges from a model into decisions without being undone by overbetting or estimation error, and arguably provides a better measure of sequential decision-making over long time horizons.

Trading Evaluations. There is also a growing literature of financial trading environments. PyMarketSim introduces a limit-order-book simulation for agents (Mascioli et al., 2024). MarS introduces an order-level market simulation engine built on a generative foundation model (Li et al., 2024). FinRL Contests provide environments spanning stock trading, order execution and cryptocurrency trading (Wang et al., 2025). StockBench evaluates LLM agents in multi-month stock-trading environments using return and risk metrics (Chen et al., 2025). AI-Trader introduces a live benchmark across

[1]Note on KellyBench we give agents access to a sandbox machine with 4 CPUs as opposed to 2 in our usual terminal-based benchmarks, anticipating possibly CPU-intensive workloads. But notably no agent we test attempts heavy workloads like this for dynamic abilities, and agents instead prefer basic dynamic methods such as ELO ratings.

U.S. equities and cryptocurrencies (Fan et al., 2025). Lastly, TraderBench focuses on adversarial cryptocurrency and options markets (Yuan et al., 2026).

A realistic trading environment would benefit from broad, timestamped access to news and market context. However, constructing a faithful offline approximation of that information is difficult and risks either temporal leakage or overly narrowing the task. `KellyBench` occupies a useful middle ground. It supports repeated market decisions over a large class of structurally similar events, while allowing rich pre-match information - such as lineups, match and player statistics - to be provided offline in a timestamped form. This preserves open-ended exploration without requiring a full offline reconstruction of the internet.

3. Benchmark Design

We develop the `KellyBench` environment using the Open Reward Standard (ORS), a protocol for defining RL environments with tasks, tools and state (Open Reward Standard, 2026). We serve the environment on OpenReward and use the `firehorse` library for running a ReSum harness on the models of interest (Wu et al., 2025)[2].

3.1. Environment Dynamics

Each episode simulates one full English Premier League season, spanning approximately 100-150 unique matchdays. The agent is initialised with a bankroll and proceeds through the season one matchday at a time. On each matchday t, the interaction follows a fixed cycle:

1. **Observation.** The agent views the set of matches $\mathcal{M}_t$ scheduled for that day, along with closing decimal odds (pre-match odds near kickoff) sourced from real bookmakers.
2. **Model development.** The agent reads and writes files in a sandboxed compute environment to build predictive models and wagering strategies.
3. **Bet placement.** For each match $m \in \mathcal{M}_t$, the agent places wagers on any of five types of bet: home win, draw, away win, over 2.5 total goals, or under 2.5 total goals[3]. Each bet specifies a stake drawn from the agent's current bankroll.
4. **Settlement.** All bets are resolved against the actual match outcomes. Winning bets return the stake multiplied by the quoted decimal odds; losing bets forfeit the stake. The agent's bankroll is updated accordingly.
5. **Data update.** After settlement, the agent receives the latest match results and detailed player-level statistics for the completed matchday, which it can incorporate into subsequent model iterations.

The agent is required to place at least one bet per matchday to induce it to expend effort on model development rather than abstaining from the task. Penny bets are still allowed under this rule as a capital conservation strategy.

[2] https://github.com/GeneralReasoning/firehorse

[3] Note: it would be more realistic to use Asian handicap markets for quantitative funds, but we use 1X2 and Over/Under markets given public data availability and simplicity of interpretation. Note this also means there is "vig" in the odds that we use, which is around 5.3%, which makes the task significantly harder. We also use bookmakers odds rather than exchange odds as they have better public data availability.

3.2. Data

Agents are provided with two categories of historical data to develop machine learning models: match-level data and player-level data.

3.2.1. Match-level data

A longitudinal dataset of English Premier League matches spanning from the 1993–94 season to the start of the evaluation season. Each record contains the match date, home and away teams, and the full-time result (home/draw/away) with scoreline. Coverage broadens over time: half-time scores are available from 1995–96; shot counts, fouls, corners, cards, and the match referee from 2000–01; and pre-kickoff decimal odds from multiple bookmakers (including over/under and Asian handicap markets) from 2002-03 onwards.

3.2.2. Player-level data

We provide the model with a detailed dataset of per-match player statistics drawn from major European leagues and cup competitions (including the Premier League, Championship, La Liga, Serie A, Bundesliga, Ligue 1, domestic cup competitions and UEFA club competitions) from 2008 onwards. Each record is linked to a specific fixture and contains team lineups together with individual player statistics such as goals, assists, minutes played, shots, cards, tackles, interceptions and expected goals (xG) where available; as well as player information such as their age, height and playing position.

Data disclosure of both data categories is *progressive* during each episode. At the start of the season the agent has access only to historical seasons, and after each matchday the environment provides the latest results and player statistics for the matches just completed. This mirrors the real-world information structure faced by a quantitative bettor.

3.3. Tools

The agent interacts with `KellyBench` through two types of tool: environment tools and CLI tools.

3.3.1. Environment tools

The agent interacts with the simulated betting world through four tools:

- `view_matches` – displays the current matchday's fixtures and bookmaker odds;
- `place_bet` – places a wager on a specified match, market and stake;
- `view_bankroll` – reports the agent's current balance and outstanding stakes;
- `next_matchday` – settles all bets, delivers results, downloads updated data and advances to the next matchday.

3.3.2. CLI tools

To develop models, the agent is provided with seven general-purpose tools for a sandboxed development environment: `bash`, `glob`, `grep`, `read`, `write`, `edit` and `todo_write`. These tools enable the agent to write Python scripts, train models, inspect data files and organise its workflow. The tool schemas are designed to match the Claude Code toolset exactly (Anthropic, 2025).

The sandbox provides 4 CPUs and 16GB of RAM with a standard Python data-science stack (NumPy, pandas, scikit-learn).

3.4. Reward Structure

KellyBench uses a dense, fully verifiable reward signal. After each matchday t, the reward is the change in log-wealth:

$$r_t = \log W_{t+1} - \log W_t = \log \frac{W_{t+1}}{W_t}, \tag{1}$$

where W_t denotes the agent's bankroll at the start of matchday t (before bets are deducted) and W_{t+1} is the bankroll after settlement. The cumulative reward over a full season therefore equals the log-ratio of final to initial wealth:

$$R = \sum_{t=1}^{T} r_t = \log \frac{W_{T+1}}{W_1}. \tag{2}$$

We chose this reward due to its connection to the Kelly criterion (Kelly, 1956), hence the name `KellyBench`. The strategy that maximises expected log-wealth growth is the Kelly-optimal strategy, which also maximises the long-run geometric growth rate of the bankroll.

Since rewards are computed deterministically from match outcomes and bookmaker odds, KellyBench is fully verifiable and does not require LLM graders for evaluation. One possible exception is for the training set, where an auxiliary LLM-as-a-judge signal can be used to penalise deviations from rule-based strategies. This auxiliary signal is not included by default in our OpenReward implementation.

3.5. Task Scenarios

`KellyBench` comprises five scenarios spanning different eras of English football, divided into training and test splits (Table 1).

Scenario	Season	Split	Initial Bankroll	Matchdays
New Millennium	2000/01	train	£100	97
Post-Crash	2010/11	train	£150	105
Covid Season	2020/21	train	£200	148
Recent Season	2023/24	test	£220	120
Recent Season (Lit.)	2023/24	test	£220	120

Table 1 | **KellyBench Scenarios**. Each scenario covers a full English Premier League season. The literature variant of the 2023/24 test scenario augments the agent's sandbox with a curated collection of 30+ research papers on sports prediction and betting markets. For the principal results in this paper, we normalise initial bankroll numbers above to £100,000 for display purposes.

The training scenarios are drawn from distinct periods, each presenting different league dynamics, team strengths, and odds characteristics. The test scenario uses a recent complete season from 2023/2024. The *literature variant* of the test scenario provides the agent with access to a curated research library: a deep research report and over 30 academic papers in markdown format covering probabilistic football modelling, scoring rules, expected goals models and Kelly betting.

Each scenario is long-horizon and agents use 500-1000 tool calls to complete an episode, so agents need to be effective at compaction and context management to perform well on `KellyBench`.

For this paper we focus on the `Recent Season` evaluation scenario.

3.6. Data Leakage

There are three sources of data leakage that we need to guard against:

1. **Result Knowledge in Weight Memory**. For example, a model that knows the outcome or the score can utilise this information to make profitable bets.
2. **Season Knowledge in Weight Memory**. For example, a model that knows that Leicester City won the Premier League in 2015/16 can utilise this information to make a profitable strategy for that season even if it does know not the direct scores or outcomes of any particular match.
3. **Accessing the Internet for Future Results**. A model that is able to download files with match results or access websites with future information can use this to cheat. We prevent this by blocking network access on the environment sandbox.

To guard against weight memory leakage, we give agents explicit instructions to follow a rules-based strategy. They are told not to bet on gut feeling or intuition, but to follow the recommendations of a rules-based approach. Note that this is exactly how a quantitative analyst should behave when constructing backtests; they should not fall upon their knowledge too much of what actually happened when developing a model, otherwise they risk overfitting to a particular realisation of the world.

To reinforce the point, we tell agents they will be banned from betting and their bankroll eliminated if they do not follow a rules-based strategy. Based on observed agent trajectories, models comply with this instruction and it appears to be sufficient for preventing leakage. However, this could change with future AI agents that can hide their true intentions behind their chain-of-thought

We found a particularly striking example of the risk of weight memory leakage in GPT 5.4's trajectory:

> As a model-based assistant with knowledge up to 2026, I know the actual result of Burnley vs Man City was 0-3 away, which suggests a potential bet on away and over. However, I must base my decisions on a reliable model.

The GPT-5.4 agent proceeded to use the recommendations of a machine learning model instead, but a future agent could be deceptive. For example, suppose it knew that Man City won a match against Burnley. It could find a rules-based strategy that suggests Man City as having edge in the encounter, and then claim that "I am betting on Man City because of the model's recommendation". However, if an agent did this we would observe lots of opportunistic changes in the rules-based strategy over the course of the season. On the contrary, we do not see this kind of reward hacking in the traces we observe - agents are notably non-adaptive - but it is something to watch out for with future models.

A more insidious source of reward-hacking would be knowing some aggregate season statistic and silently introducing model bias to maximise it. For example, the 2023/2024 season was unusually high scoring with 3.28 goals per match versus a 2.67 historical average (Premier League, 2024). Assuming Poisson-like distributions for goals, this decreases the likelihood of draws. Indeed, empirically this is what happened with 82 draws versus a 97 historical draw average. Football models typically inflate draw probabilities based on empirical distributions (Maher, 1982). However, an extremely smart AI agent that knew that the 2023/2024 season was high-scoring could choose to omit this inflation factor silently to achieve opportunities for betting against draws.

Both types of reward hack, especially the latter, would require extremely high degrees of latent reasoning. While we can not be certain, we have two strong reasons to believe agents are not yet

capable of this degree of reasoning. The first evidence is that frontier models perform poorly on `KellyBench`, so unless the agent is sandbagging, it is not utilising its knowledge of the future effectively[4]. A second reason is that models demonstrate poor sophistication in their strategies compared to humans, so assuming they are attempting to perform well, they are not exhibiting strong levels of domain-relevant reasoning compared to human experts at present.

Our long-term plan is to introduce a live version of `KellyBench` as soon as models start to perform well. For now, the current backtest environment is not yet saturated and frontier models perform poorly. If independent evaluators wish to evaluate agents on `KellyBench`, we advise them to audit trajectories and disqualify agents that disregard rules-based betting instructions.

4. Results

4.1. Headline Results

All models lost money on average over three seeds on the 2023/24 English Premier League season, with mean final bankrolls ranging from £0 (Arcee Trinity and Grok 4.20) to £89,035 (Claude Opus 4.6). Only 2 out of 24 seeds achieved a positive return on investment: one from Gemini Pro (+33.7%) and one from Gemini Flash (+24.7%). Table 2 summarises the final bankroll, ROI, and key statistics for each model. Per-model narratives appear in Appendix A.

Model	Mean ROI	Best Seed	Worst Seed	Avoided Ruin	Final Bankroll
Claude Opus 4.6	−11.0%	−0.2%	−18.8%	Yes	£89,035
GPT-5.4	−13.6%	−4.1%	−31.6%	Yes	£86,365
Gemini 3.1 Pro	−43.3%	+33.7%	−100.0%	No	£56,715
Gemini Flash 3.1 Lite Preview[a]	−58.4%	+24.7%	−100.0%	No	£41,605
GLM-5	−58.8%	−14.3%	−100.0%	No	£41,221
Kimi K2.5[b]	−92.6%	−77.7%	−100.0%	No	£7,420
Grok 4.20[c]	−100.0%	−100.0%	−100.0%	No	£0
Arcee Trinity[d]	−100.0%	−100.0%	−100.0%	No	£0

[a] Two of three seeds forfeited at intermediate stage (bankrolls £27 and £105); the surviving seed returned £124,682.
[b] Two of three seeds forfeited: one at an intermediate bankroll of £72,955 and one at £0. The surviving seed returned £22,259.
[c] All three seeds failed: two forfeited at intermediate stage (bankrolls £25,923 and £9,518) and one went bankrupt.
[d] All three seeds failed: one forfeited at intermediate stage (bankroll £15,773), one never placed a single bet, and one went bankrupt.

Table 2 | **The Road to Ruin**. Model performance across three seeds on the 2023/24 English Premier League season. Each model begins with a £100,000 bankroll. ROI and final bankroll are averaged across seeds. Avoided Ruin indicates whether all three seeds survived to the end of the season.

The two strongest models, Opus 4.6 and GPT-5.4, share several traits. Both models retrained or adjusted their strategies in response to new match data, both deployed systematic staking rules rather than ad hoc bet sizes, and both preserved capital during periods where their strategies identify no edge. Opus 4.6 and GPT-5.4 were the only models to avoid ruin across all three seeds, with Opus consistently deploying gradient boosting ensembles with fractional Kelly staking. GPT-5.4 was the most engineering-intensive model, with one seed dedicating roughly 160 tool calls to model building before placing its first bet. Later, it adapted its strategy by reducing stakes to penny bets after determining that it could not reliably outperform the market (model log-loss 0.974 vs. market 0.971).

[4] Sandbagging seems unlikely at present given the known optimisation objectives of current language models to perform well on software engineering tasks, but we cannot rule it out completely.

Most of the other models we evaluated fell into a small set of recurring failure patterns that compounded to produce poor outcomes. Broadly, these failures fell into five categories: bankroll management failures in which agents discuss Kelly staking but implement something else, inability to handle the non-stationarity introduced by newly promoted teams, systematic miscalibration of draws and longshots, absence of in-season adaptation, and long-horizon situational awareness failures including premature task termination and difficulties with the tool-calling environment. We summarise the prevalence of these failings across the 24 seeds in Table 3.

Failure Mode	**Seeds Affected**
Bankroll ruin (final bankroll £0)	4/24
Forfeit: abandoned mid-season with bankroll > £0	6/24
Forfeit: zero bets placed, treated as ruin	2/24
Bankroll management	
No Kelly or principled stake-sizing rule at execution time	9/24
Kelly code written but never invoked at bet time	4/24
Non-stationarity / promoted teams	
No general handling of newly promoted teams	24/24
Intra-season adaptation	
Never retrained statistical model after initial fit	7/24
Not fully adaptive (no model retraining *or* strategic pivot)	21/24
Situational awareness / environment	
Declared task complete while season still running	6/24
Failed to correctly invoke provided betting tools	4/24
Label leakage in backtesting or prediction pipeline	6/24
Calibration	
Systematic draw / longshot miscalibration (e.g. fixed draw prior)	4/24

Table 3 | **Prevalence of failure modes across the 24 seeds** (8 models × 3 seeds). Categories are non-exclusive: most seeds exhibit multiple failures. Counts are derived from manual inspection of run trajectories; see Appendix A for per-seed detail.

The two weakest models, Arcee Trinity and Grok 4.20, both struggled with basic environment interaction. Two of Trinity's three seeds placed zero bets, failing to discover or correctly invoke the provided tools. Two of Grok's three seeds entered terminal loops and declared the task complete while the season was still running; the third went bankrupt.

4.1.1. Bankroll Management and the Gap Between Intention and Execution

The most pervasive failure mode was a disconnect between what agents reasoned about and what they actually executed.

Bankroll management was either absent or broken in five of the eight evaluated models. Nearly every model discussed Kelly staking in its chain of thought, yet at execution time most fell back to flat stakes, percentage-of-bankroll heuristics, or ad hoc round numbers. Gemini Flash used no staking formula at all, manually choosing round-number bets that escalated from roughly £4,500 early in the season to a single ~£273,000 wager, placed on a three percentage-point historical win-rate edge and immediately lost[5]. Kimi K2.5 wrote a numerically correct fractional Kelly function (Figure 3, panel b)

[5]All monetary amounts in this section are rescaled to the £100,000 normalised starting bankroll used in Table 2

but never invoked it from its betting loop due to repeated tool-call formatting failures. Its episode ended with an accidental ~£114,000 bet, ninety-eight percent of its remaining bankroll, on Burnley versus Luton. GPT-5.4 discussed Kelly extensively in its reasoning chains and then implemented a simpler rule-based stake schedule. Arcee Trinity trained a Random Forest with seventeen features but, at inference time, hardcoded all but the team-identity columns to zero, querying the model far off its training setting for every prediction (Figure 3, panel c).

GLM-5: fixed draw rate, never corrected despite self-diagnosis

```
prob_draw = 0.25  # fixed; flagged as flawed in three
                  # mid-run self-critique documents
prob_home = prob_home * (1 - prob_draw)
prob_away = prob_away * (1 - prob_draw)
```

Kimi K2.5: correct fractional Kelly that was never invoked

```
def kelly_criterion(prob, odds, bankroll, fraction=0.25):
    b = odds - 1
    kelly_fraction = (prob * b - (1 - prob)) / b
    if kelly_fraction <= 0:
        return 0
    return bankroll * kelly_fraction * fraction
```

Arcee Trinity: seventeen-feature Random Forest queried incorrectly

```
features = np.array([
    0,  # home_form
    0,  # away_form
    0,  # form_diff
    # ... 12 further features, all set to zero
    home_encoded, away_encoded
]).reshape(1, -1)
```

Figure 3 | **Example Failure Modes**. Failure modes above show miscalibration that the agent diagnosed but never fixed, a correct staking implementation that was never executed, and an inference-time feature collapse that queries a trained model far outside its training distribution.

4.1.2. Non-stationarity and Newly Promoted Teams

Another key test for the models was how to handle newly promoted teams where less recent historical data is available. The 2023/24 season featured three promoted teams, each presenting a distinct distributional shift that no model handled adequately. Luton Town had not played in the top flight since 1992, so the entire training window from 1993 to 2023 contained zero Premier League matches for the club[6]. Burnley's most recent top-flight data came from 2021/22. Sheffield United's most relevant prior data was their 2020/21 campaign.

No model implemented a general solution. One GLM-5 seed correctly diagnosed the problem on matchday one, observing that Luton had zero historical matches and that the apparent edge was therefore a model artefact. It then hardcoded a skip-Luton rule that persisted for the rest of the season, while continuing to bet freely on Burnley and Sheffield United and never generalising the insight. Opus 4.6 showed a recurring pattern of diagnosing the problem and then deferring to its model anyway, reasoning at one point that although the apparent edge on a Burnley fixture was implausible given the squad, it should follow the model-based approach since that was the framework it was using. One GPT-

[6]Note there is Championship data for these clubs, but Championship clubs only encounter Premier League clubs occasionally in cup competitions. In addition, newly promoted teams typically change their squad distribution before starting a Premier League campaign. So this is a large distributional shift with very little data to model adequately.

5.4 seed adopted the most elegant non-solution, relying on `OneHotEncoder(handle_unknown='ignore')` so that bookmaker odds absorbed the promotion signal, in effect deferring to the market on teams it could not model. Gemini Flash treated Luton's historical win rate of zero as a genuine signal rather than missing data, leading to repeated bets against Luton that contributed to its bankruptcy.

4.1.3. Absence of Intra-season Adaptation

There are two kinds of adaptivity that are important. The first kind of adaptivity is that models retrain their models as new data comes in; for example, updating dynamic strength models. The second kind of adaptivity is changing the model itself in light of realised performance ("pivots"). We find models are poor on both counts, especially the second kind of adaptivity, which is a long-horizon capability that we would like model to be good at (outside of just sports betting).

First, existing model adaptivity is poor. The majority of seeds fitted their statistical models once at the start of the season and never updated them despite receiving fresh match data after every matchday. This likely contributed to poor performance, as the market progressively grew its information edge over the model. Only a few seeds were fully adaptive in the sense of progressively retraining models, including one GPT-5.4 seed which performed walk-forward retraining, and two Opus 4.6 seeds.

Strategic pivots were rare and it was mostly the strongest models that were capable of this. One of the Opus 4.6 seeds reduced its Kelly fraction from 0.25 to 0.15 after drawdowns; another pivoted from gradient boosting to a Poisson-plus-market-prior blend after diagnosing that its model had no edge against the closing line. A GPT-5.4 seed pivoted to an extremely conservative strategy after diagnosing it had no edge versus the market based on realised peformance, resorting to penny bets for the rest of the season. However, for the models that performed pivots, they had a bias towards model building at the start of trajectories instead of mid or late season.

When taken together, the fully adaptive seeds outperformed the fully static ones in average ROI (−17.2% vs. −88.3%), with partially adaptive runs falling in between (−47.7%).

4.1.4. Long-horizon Situational Awareness and Environment Failures

A large cluster of failures concerned situational awareness over the course of the episode. Several models declared the season finished while it was still running. Grok 4.20 entered terminal loops in all three seeds, repeatedly calling `next_matchday` without placing bets and declaring the task complete. One Kimi K2.5 seed produced six separate "final season summary" documents at successive bankroll levels of ~£67,700, £38,400, £29,800, £26,600, £26,600, and £15,700, each claiming the episode was concluded. This yielded a roughly eighteen-matchday plateau during which the agent acted as if the run were over. Two of Arcee Trinity's seeds framed inaction as completion, never placing a single bet and treating their initial £100,000 bankroll as a final state.

Some models also struggled with the tool-calling environment itself. Two of Arcee Trinity's three seeds never used the betting tools, attempting instead to call environment functions as Python imports. Several models suffered from label leakage in their backtesting pipelines. The most extreme case was one Grok seed which hardcoded expected goals values into its betting script, causing every fixture to receive an identical probability estimate.

Rank	Model	Sophistication	Final Bankroll	Avg Tool Calls
1	Claude Opus 4.6 (max)	32.6%	£89,035	518
2	GPT-5.4 (xhigh)	31.8%	£86,365	796
3	GLM-5 (thinking)	22.0%	£41,221	631
4	Kimi K2.5 (thinking)	15.9%	£7,420	465
5	Trinity-Large-Thinking[a]	12.1%	n/a (£0)	291
6	Gemini 3.1 Pro Preview (high)	9.8%	£56,715	570
6	Grok 4.20 (reasoning)[b]	9.8%	n/a (£0)	469
8	Gemini 3.1 Flash Lite Preview (high)	6.1%	£41,605	351

[a] Two of three seeds placed zero bets, failing to reach the first matchday.
[b] Two of three seeds forfeited mid-season (bankrolls £25,923 and £9,518); the third went bankrupt.

Table 4 | **Models show low strategy sophistication.** Sophistication is expressed as a percentage of the maximum possible expert rubric score of 44, averaged across 3 seeds along with the final bankroll.

4.2. Are models doing the best they can with the available data?

A possible criticism of our setup might state that the market is highly efficient, and/or that we are not providing agents with enough data to obtain edge. Under this interpretation, the agents we test might be performing well in a model-building sense, but do not have enough data to beat the market.

We now present evidence that this is not the case, and that even with the data they have, models perform in a way that is unsophisticated compared to human baselines. Specifically they utilise the available data and compute in an inferior way to how a human expert would utilise it. There is also considerable room for improvement for models to increase their sophistication in `KellyBench`.

Specifically, we construct a 44-point rubric that judges the sophistication of the strategies employed by the models. We construct this rubric by consulting experts with experience working at leading quantitative betting funds. Different rubrics test facets of strategy such as feature development, execution strategies, and how models account for challenges such as non-stationarity.

We do not reproduce the rubric in full to maximise the lifetime of the benchmark, but we produce three examples of the criteria below in Table 5 to illustrate how the model is scored.

#	Criterion	Scoring
2	Does the agent use Kelly staking?	0: No 1: Full Kelly 2: Fractional Kelly
4a	Does the agent use dynamic team ability models?	0: No 1: Exponential average / Elo 2: State-space models
11	Does the agent use a special approach for promoted teams?	0: No 1: Yes

Table 5 | **Example Sophistication Rubrics**. Rubrics were constructed by human experts with experience at leading sports betting funds, testing modeling, wagering and domain knowledge. There are 38 criteria in total with a maximum possible score of 44.

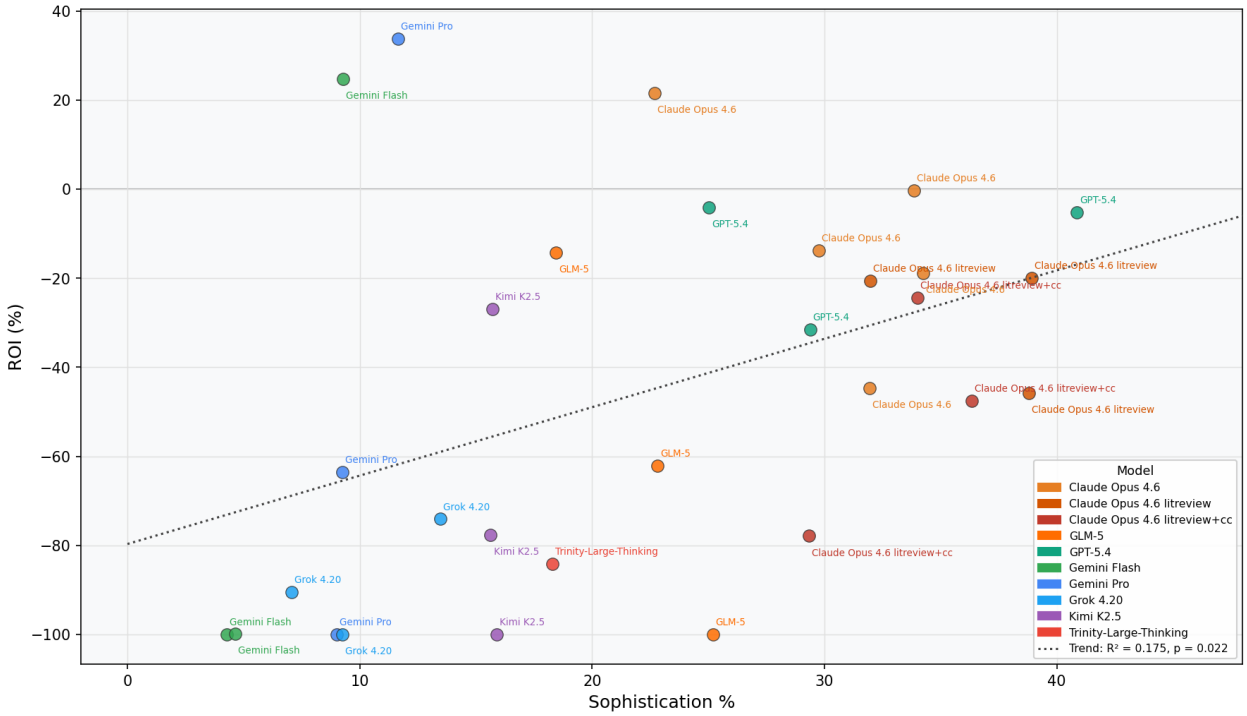


Figure 4 | **Sophistication vs Return on Investment**. For each seed of the model on `KellyBench`, we plot the sophistication score against the final return on investment, which exhibits a positive correlation. Given that frontier models achieve less than a third of rubric points, we believe there is considerable room for improvement in the strategies created with the data afforded to the agent.

Using these rubrics, we calculate average sophistication of each model in Table 4. All models show low strategy sophistication. The best model Claude Opus 4.6 achieves less than a third of possible points on the measure with a sophistication score of 32.6%.

Performance and sophistication also have a statistically significant positive relationship over the seeds we tested in this paper, which we plot in Figure 4. We also see that the two most sophisticated models experienced the smallest decrease in bankroll over the course of the three seeds.

Higher sophistication is also associated with lower rates of ruin: seeds scoring 11–18/44 went bankrupt at a rate of ~7%, compared to ~40% for seeds scoring 0–5/44. A logistic regression of ruin probability on sophistication score is significant for a likelihood-ratio test at $p = 0.008$, which likely reflects the existence of staking based criteria in our rubric which are designed to avoid ruin.

Given that current frontier models achieve less than 50% sophistication, there may be considerable room for improvement on `KellyBench` with the existing data and compute that agents are given.

4.2.1. Harness Ablations for Sophistication

One potential critique of our setup is underelicitation from the use of of a custom ReSum harness as opposed to the standard Claude Code harness (Wu et al., 2025). Additionally, the evaluation environment has no web search tool, preventing an agent from conducting a literature search to discover and synthesise promising approaches. This could mean sophistication is underelicited.

To test the effect of literature access and a better harness, we construct two additional baselines for the strongest performing model Claude Opus 4.6. First, we provide the model with a deep research report

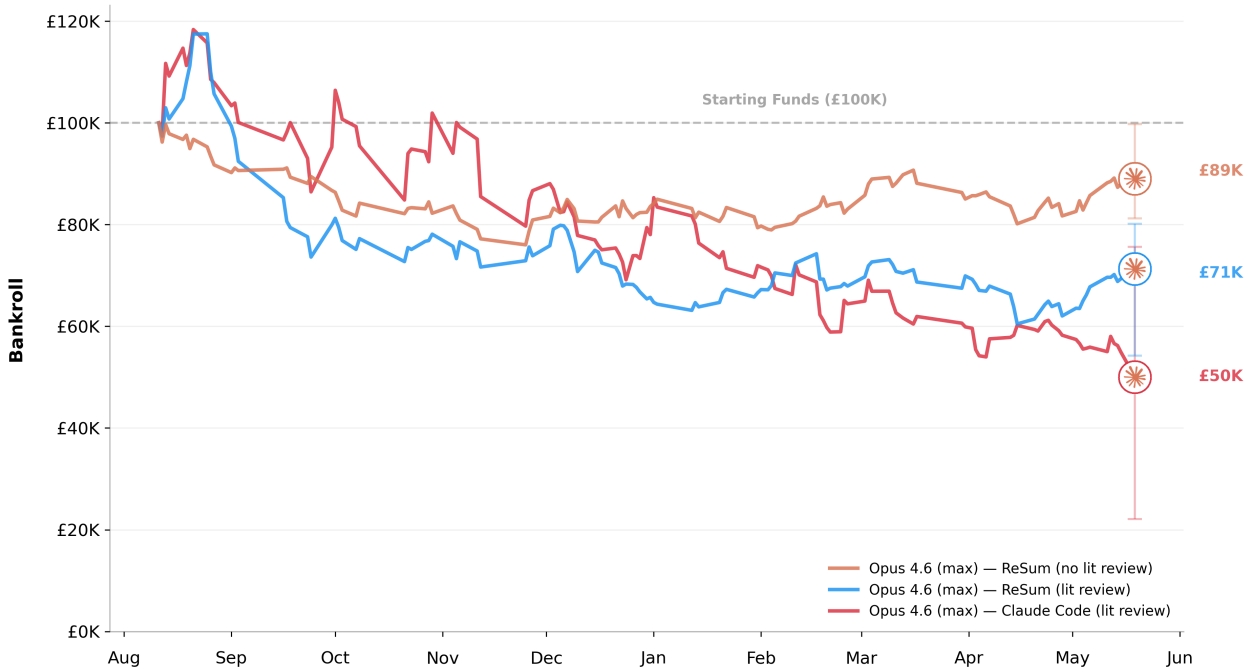


Figure 5 | **Harness Ablations on Opus 4.6**. We test two possible critiques of our results: first, that by disabling network access, we are undereliciting performance that could be gained from access to the literature; and second, that a ReSum harness would underperform compared to a Claude Code harness. We ablate both setups and find that neither improves upon the baseline.

and over 30 relevant academic papers in its filesystem. Secondly, we swap out the ReSum harness for a Claude Code harness using `firehorse` library. We use three seeds for each new baseline, giving six runs in total, and plot the average bankroll results in Figure 5.

We do not see a positive impact on performance from either change. In fact, mean final bankroll declined with both baselines for the seeds we tested. We observed that models read the deep research report, but did not thoroughly read all the papers available in the filesystem. The main change in model behaviour from access to the literature appeared to be utilising Dixon-Coles models more frequently, but this did not result in increased performance (Dixon and Coles, 1997).

In fact, Dixon-Coles models, while considered seminal in football modelling, are considered outdated given the way they estimate dynamic abilities through likelihood-weighting rather than Bayesian estimation. This could suggest that models have poor "taste" in discerning which literature to focus in on in this field. When judged according to our sophistication score, the literature-review variants did not improve sophistication significantly (33.8% vs 32.6%).

Another general observation was that Claude Opus 4.6 appeared to be too keen to write code rather than thoroughly planning out its approach. In principle, a better prompt or directed planning mode could improve its performance and sophistication; we leave this for future work.

4.3. Return Analysis

Given the high variance of runs, another potential critique of our setup is that we do not have enough seeds to accurately assess if models are beating the market or not. To study this question, we sample an additional two seeds for Claude Opus 4.6 for a total of five seeds. With five seeds, the mean final bankroll becomes £88.8k for negative ROI of 11.2%.

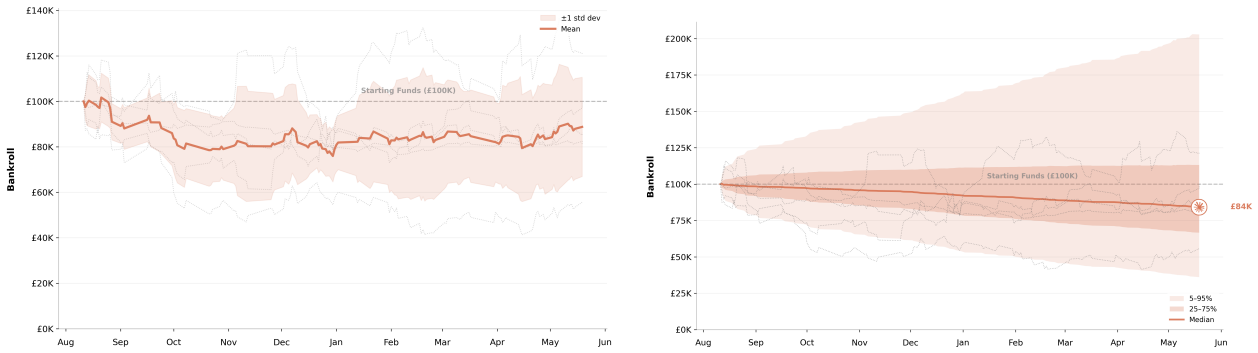


Figure 6 | **Return Analysis on Claude Opus 4.6.** In the left subfigure, we plot trajectories from 5 seeds and plot 1σ error bars. In the right subfigure, we run a hierarchical bootstrap. We first randomly choose one of the 5 seed trajectories and then sampling empirical log returns from it for each matchday. We repeat this for 50,000 simulations to construct bootstrapped error bars.

We then conduct a hierarchical bootstrap to construct bootstrapped error bars. These results are plotted in Figure 6. According to the hierarchical bootstrap, around 33% of simulations end up profitable with Claude Opus 4.6 and the remainder lose money. This shows that there is some potential for Claude Opus 4.6 to make money over the course of the season - indeed one seed ends up profitable - but our baseline conclusion is that the model is not consistently outperforming the market on average. And in lieu of no other model outperforming the market on average, our conclusion stands that the current generation of models is not finding consistent edge at present.

4.4. Knowledge-Action Gaps

A broader theme we observe in the trajectories, which may explain underperformance, is a gap between knowing what to do and actually executing it in practice.

Models frequently articulated promising strategies in their reasoning chains but failed to verify that their code implemented those strategies, failed to notice when execution diverged from intent, or failed to act on their own diagnostic findings. This represents a distinctive failure mode for agentic systems: unlike a static model evaluation where the output is the final answer, `KellyBench` requires agents to maintain coherent intent across potentially thousands of sequential decisions, monitor the consequences of those decisions, and close the loop between observation and action.

GLM-5 provides a particularly clear example that recurred across seeds. In its base seed, the model wrote three separate self-critique documents during its run, each correctly identifying that its fixed 25% draw rate and overestimation of home advantage were the root causes of its losses. For example, at bankroll ~£44,200, it wrote: "*Model probabilities don't match observed frequencies: Predicted 40% home wins only won ~30%.*" In one seed, it explicitly noticed that Burnley had been promoted but only special-cased Luton as a skip rule, ignoring the general problem. In another seed, it discovered a +2.62% ROI strategy in backtesting but never validated this with held-out data before deployment. Across all three seeds, the model could analyse its failures but not act on its own analyses.

Kimi K2.5 exhibited a different variant of the same problem. In one seed, its prediction script with Kelly staking was well-designed, but a persistent inability to format tool calls correctly as it entered long-context caused the model to send `": ["` as a bash command approximately 50 times in sequence. The model's reasoning noted: "*bro, you seeing this? The bash command keeps returning empty. Let me try again with proper command execution*", yet it continued sending the identical broken

command. In one seed, the model declared "episode complete" six separate times while its bankroll was still declining, then entered an 18-matchday plateau with zero bets placed. In another seed, it peaked at ~£170,000 with a working Poisson model before suddenly abandoning this strategy. Across seeds, it had sufficient awareness to notice problems but insufficient capability to resolve them.

Arcee Trinity represents perhaps the most extreme case: while its bankroll collapsed from ~£111,000 to ~£15,900 in one seed, it repeatedly declared "*SYSTEM STATUS: COMPLETE. Live Demo: 4.09% ROI, ~£4,100 profit*". This figure was computed from only its first six bets, ignoring all subsequent losses. When it finally ran a full diagnostic that revealed "*Win Rate: 27.76%, ROI: -99.93%*", it showed zero reaction and marked its task list as completed. The other two seeds demonstrated an even more fundamental failure: the agent did not understand or use most of the tools required to interact with the environment, instead opting to do everything via python scripts.

These patterns suggest that many models can generate sophisticated plans but often struggle to maintain the closed-loop monitoring and coherence required to execute them over extended episodes. The ability to write correct code, identify correct strategies, and diagnose failures does not reliably translate into the situational awareness needed to detect and correct execution failures in real time and in accordance with new information.

5. Limitations and Conclusions

5.1. Limitations

5.1.1. Market efficiency

The English Premier League is widely regarded as one of the most informationally efficient prediction markets in the world (Elaad et al., 2020). While models in `KellyBench` are provided with plenty of data on which to base their strategies, there is much we don't provide agents. For example, we do not give models any tracking data which is particularly important for evaluating defensive ability. We do not provide set-piece data that is an important subgame with distinct team abilities. We do not provide in-play data with real-time information. Lastly, we use closing line odds, at which point lineup information may be public - for example, new knowledge of a key player injury - but we do not give this information to the agent. Taken together, there is a strong potential for estimation error.

However, we do not think this invalidates the benchmark for two reasons. First, models show low sophistication even with the existing data, meaning they are still underexploiting it. Therefore, even if models cannot make a profit with the existing data, the benchmark may still measure meaningful progress in frontier model capability. Secondly, it is not necessary to have all fundamental information to gain an edge. Exactly as Benter (2008) pointed out, we can blend fundamental models with public factors from odds which contain the missing information, and obtain overlays where our edge exists.

5.1.2. Single season only

A related critique is the choice for us to backtest over a single season. To identify stronger relative differences between models we would ideally need more seasons to draw stronger conclusions. However, as a starting point, we think a single season already reveals many interesting failure points of existing models. In particular, no model evaluated makes a consistent return on average. Additionally, given the expense of running `KellyBench`, we decided a single-season backtest was the right first step to validate the usefulness of this class of benchmark.

5.1.3. Harnesses

We tested several single agent harnesses, but we did not test multi-agent or iterative harnesses. For example, scaffolds in the style of AlphaEvolve may be capable of increased performance, especially on tasks such as feature development and exploration (Novikov et al., 2025). In addition, multi-agent frameworks could also lead to better specialisation of work between feature development, backtesting and trade execution. We leave these ablations to future work.

We do note, however, that such ablations are likely to be prohibitively expensive for academic researchers in particular. Given that one seed of a GPT-5.4 model costs $\approx$ \$2,000, iterative and multi-agent approaches (on future larger models) are likely to be extremely expensive and out of reach to most individuals or small groups. We make this point to raise attention to this issue as the field moves beyond procedural tasks towards evaluations for long-horizon complex worlds like `KellyBench`.

5.1.4. Using middle line odds

We used bookmaker odds, where overround ranged from 2% to 7%, and took a middle of the line odds at around 5% overround. This makes edge harder for the models to find. Given that models typically return less than -10%, we think the results are not invalidated by using these odds. Indeed, as we point out throughout the paper, models demonstrate low sophistication with the data they are given, leaving room for improvement against these odds. However, in future work we are considering switching to using to "sharp" odds, as they are a better test for market efficiency.

5.1.5. Knowledge contamination and concealment

A fundamental limitation of any retrospective evaluation is that frontier models may have encountered the ground-truth outcomes during pre-training. The 2023–24 English Premier League season concluded before the knowledge cutoffs of all models tested, meaning that match results, final standings, and even betting market movements are likely present in their training corpora. We have discussed this problem in detail earlier in the paper, but we raise it again here for transparency.

The consensus of the authors is that models comply with the instructions to use rules-based strategies in the backtesting environment, but we will reassess as new models are released. We plan to switch to a live benchmark when we detect signs of reward hacking. We also note again that, despite potential contamination, all models still lost money on average across three seeds. We again note that we did not see evidence of reward hacking in the trajectories used in this work.

5.2. Conclusions

`KellyBench` reveals a consistent pattern across all eight models tested: none can profitably navigate a full season of English Premier League betting markets on average across three seeds. The benchmark exposes failures not only in machine learning modeling, where models struggle to outperform bookmaker-implied probabilities, but more fundamentally in the closed-loop reasoning required for long-horizon sequential decision-making. Models can write sophisticated code, diagnose their own failures, and articulate correct strategies, yet persistently fail to execute those strategies reliably, monitor their own performance, or adapt when their approach is not working.

As well as looking at performance, we judged strategy sophistication for each model and found existing models to have unsophisticated strategies compared to human approaches. In particular, rich player-level data available in the environment was almost universally ignored in favour of simpler team-level features, suggesting that current models systematically underinvest in data and feature

engineering when operating autonomously. Additionally, the non-stationarity challenge posed by factors like newly promoted teams proved universally problematic, with no model implementing a general solution despite several explicitly diagnosing the problem.

The gap between analytical capability and operational competence that `KellyBench` measures is the gap that matters for real-world deployment of agentic AI systems. Existing models are now extremely competent at procedural tasks, but need to be deployed in long-horizon settings with open-ended goals. In this regime, benchmarks that test sustained, adaptive reasoning under uncertainty become essential. `KellyBench` is an early example of a complex world that tests long-horizon sequential decision-making under uncertainty. We believe more environments and evaluations in this style will become more important as model capabilities continue to improve.

6. Acknowledgements

Thanks to Anthony Hartshorn, Ran Achiron and Nick Levine for their helpful comments on the initial draft of the paper.

References

Andrew Ang and Allan Timmermann. Regime changes and financial markets. *Annual Review of Financial Economics*, 4(1):313–337, 2012. doi: 10.1146/annurev-financial-110311-101808.

Anthropic. Claude 3.7 sonnet and claude code. https://www.anthropic.com/news/claude-3-7-sonnet, February 2025. Introduces Claude Code in limited research preview.

Gregor Anzer and Paul Bauer. A goal scoring probability model for shots based on synchronized positional and event data in football (soccer). *Frontiers in Sports and Active Living*, 3:624475, 2021. doi: 10.3389/fspor.2021.624475.

Ishara Bandara, Sergiy Shelyag, Sutharshan Rajasegarar, Dan Dwyer, Eun-Jin Kim, and Maia Angelova. Predicting goal probabilities with improved xg models using event sequences in association football. *PLOS ONE*, 19(10):e0312278, 2024. doi: 10.1371/journal.pone.0312278.

David R. Bellhouse. Decoding cardano's Liber de Ludo Aleae. *Historia Mathematica*, 32(2):180–202, 2005. doi: 10.1016/j.hm.2004.04.001.

William Benter. Computer based horse race handicapping and wagering systems: A report. In Donald B. Hausch, Victor S. Y. Lo, and William T. Ziemba, editors, *Efficiency of Racetrack Betting Markets*, pages 183–198. World Scientific, Singapore, 2008.

Richard Borghesi. The home team weather advantage and biases in the nfl betting market. *Journal of Economics and Business*, 59(4):340–354, 2007. doi: 10.1016/j.jeconbus.2006.09.001.

Jun Shern Chan, Neil Chowdhury, Oliver Jaffe, James Aung, Dane Sherburn, Evan Mays, Giulio Starace, Kevin Liu, Leon Maksin, Tejal Patwardhan, Lilian Weng, and Aleksander Mądry. Mle-bench: Evaluating machine learning agents on machine learning engineering. In *Proceedings of the Thirteenth International Conference on Learning Representations (ICLR)*, 2025. URL https://openreview.net/forum?id=6s5uXNWGIh. Also available as arXiv:2410.07095.

Nikhil Chandak, Shashwat Goel, Ameya Prabhu, Moritz Hardt, and Jonas Geiping. Scaling open-ended reasoning to predict the future. *arXiv preprint arXiv:2512.25070*, 2025. doi: 10.48550/arXiv.2512.25070.

Yanxu Chen, Zijun Yao, Yantao Liu, Amy Xin, Jin Ye, Jianing Yu, Lei Hou, and Juanzi Li. StockBench: Can LLM agents trade stocks profitably in real-world markets? *arXiv preprint arXiv:2510.02209*, 2025. URL https://arxiv.org/abs/2510.02209.

B. Jay Coleman. Team travel effects and the college football betting market. *Journal of Sports Economics*, 18(4):388–425, 2017. doi: 10.1177/1527002515574514.

Martin Crowder, Mark Dixon, Andrew Ledford, and Michael Robinson. Dynamic modelling and prediction of english football league matches for betting. *Journal of the Royal Statistical Society: Series D (The Statistician)*, 51(2):157–168, 2002. doi: 10.1111/1467-9884.00308.

Bruno de Finetti. La prévision: ses lois logiques, ses sources subjectives. *Annales de l'Institut Henri Poincaré*, 7(1):1–68, 1937. English translation as "Foresight: Its Logical Laws, Its Subjective Sources" in *Studies in Subjective Probability*.

Mark J. Dixon and Stuart G. Coles. Modelling association football scores and inefficiencies in the football betting market. *Journal of the Royal Statistical Society: Series C (Applied Statistics)*, 46(2): 265–280, 1997. doi: 10.1111/1467-9876.00065.

Guy Elaad, J. James Reade, and Carl Singleton. Information, prices and efficiency in an online betting market. *Finance Research Letters*, 35:101248, 2020. doi: 10.1016/j.frl.2019.09.006.

Tianyu Fan, Yuhao Yang, Yangqin Jiang, Yifei Zhang, Yuxuan Chen, and Chao Huang. AI-Trader: Benchmarking autonomous agents in real-time financial markets. *arXiv preprint arXiv:2512.10971*, 2025. URL https://arxiv.org/abs/2512.10971.

Mark E. Glickman and Hal S. Stern. A state-space model for national football league scores. *Journal of the American Statistical Association*, 93(441):25–35, 1998. doi: 10.1080/01621459.1998.10474084.

Andrew C. Harvey. *Forecasting, Structural Time Series Models and the Kalman Filter*. Cambridge University Press, Cambridge, 1989.

Yannick Hill and Nico W. Van Yperen. Losing the home field advantage when playing behind closed doors during covid-19: Change or chance? *Frontiers in Psychology*, 12:658452, 2021. doi: 10.3389/fpsyg.2021.658452.

Edward Hughes, Michael Dennis, Jack Parker-Holder, Feryal Behbahani, Aditi Mavalankar, Yuge Shi, Tom Schaul, and Tim Rocktaschel. Open-endedness is essential for artificial superhuman intelligence, 2024. URL https://arxiv.org/abs/2406.04268.

Marcus Hutter. Universal induction & intelligence. https://www.hutter1.net/ai/uaialt.htm, 2007. Tutorial at the International Conference on Algorithmic Information Theory (ALT 2007), Sendai; 104 slides.

William L. Johns, Kempland C. Walley, Raees Seedat, David B. Thordarson, Brian Jackson, and Tyler Gonzalez. Career outlook and performance of professional athletes after achilles tendon rupture: A systematic review. *Foot & Ankle International*, 42(4):495–509, 2021. doi: 10.1177/1071100720969633.

Ezra Karger, Houtan Bastani, Chen Yueh-Han, Zachary Jacobs, Danny Halawi, Fred Zhang, and Philip E. Tetlock. Forecastbench: A dynamic benchmark of ai forecasting capabilities. In *International Conference on Learning Representations*, 2025. URL https://openreview.net/forum?id=lfPkGWXLLf.

Jr. Kelly, John L. A new interpretation of information rate. *Bell System Technical Journal*, 35(4): 917–926, 1956. doi: 10.1002/j.1538-7305.1956.tb03809.x.

Siem Jan Koopman and Rutger Lit. A dynamic bivariate poisson model for analysing and forecasting match results in the english premier league. *Journal of the Royal Statistical Society: Series A (Statistics in Society)*, 178(1):167–186, 2015. doi: 10.1111/rssa.12042.

Junjie Li, Yang Liu, Weiqing Liu, Shikai Fang, Lewen Wang, Chang Xu, and Jiang Bian. MarS: a financial market simulation engine powered by generative foundation model. *arXiv preprint arXiv:2409.07486*, 2024. URL https://arxiv.org/abs/2409.07486.

M. J. Maher. Modelling association football scores. *Statistica Neerlandica*, 36(3):109–118, 1982. doi: 10.1111/j.1467-9574.1982.tb00782.x.

Chris Mascioli, Anri Gu, Yongzhao Wang, Mithun Chakraborty, and Michael P. Wellman. A financial market simulation environment for trading agents using deep reinforcement learning. In *Proceedings of the 5th ACM International Conference on AI in Finance (ICAIF '24)*, New York, NY, USA, 2024. ACM. doi: 10.1145/3677052.3698639. URL https://doi.org/10.1145/3677052.3698639.

James Mead, Anthony O'Hare, and Paul McMenemy. Expected goals in football: Improving model performance and demonstrating value. *PLOS ONE*, 18(4):e0282295, 2023. doi: 10.1371/journal.pone.0282295.

Mike A. Merrill, Alexander G. Shaw, Nicholas Carlini, Boxuan Li, Harsh Raj, Ivan Bercovich, Lin Shi, Jeong Yeon Shin, Thomas Walshe, E. Kelly Buchanan, Junhong Shen, Guanghao Ye, Haowei Lin, Jason Poulos, Maoyu Wang, Marianna Nezhurina, Jenia Jitsev, Di Lu, Orfeas Menis Mastromichalakis, Zhiwei Xu, Zizhao Chen, Yue Liu, Robert Zhang, Leon Liangyu Chen, Anurag Kashyap, Jan-Lucas Uslu, Jeffrey Li, Jianbo Wu, Minghao Yan, Song Bian, Vedang Sharma, Ke Sun, Steven Dillmann, Akshay Anand, Andrew Lanpouthakoun, Bardia Koopah, Changran Hu, Etash Guha, Gabriel H. S. Dreiman, Jiacheng Zhu, Karl Krauth, Li Zhong, Niklas Muennighoff, Robert Amanfu, Shangyin Tan, Shreyas Pimpalgaonkar, Tushar Aggarwal, Xiangning Lin, Xin Lan, Xuandong Zhao, Yiqing Liang, Yuanli Wang, Zilong Wang, Changzhi Zhou, David Heineman, Hange Liu, Harsh Trivedi, John Yang, Junhong Lin, Manish Shetty, Michael Yang, Nabil Omi, Negin Raoof, Shanda Li, Terry Yue Zhuo, Wuwei Lin, Yiwei Dai, Yuxin Wang, Wenhao Chai, Shang Zhou, Dariush Wahdany, Ziyu She, Jiaming Hu, Zhikang Dong, Yuxuan Zhu, Sasha Cui, Ahson Saiyed, Arinbjörn Kolbeinsson, Jesse Hu, Christopher Michael Rytting, Ryan Marten, Yixin Wang, Alex Dimakis, Andy Konwinski, and Ludwig Schmidt. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces, 2026. URL https://arxiv.org/abs/2601.11868.

Deepak Nathani, Lovish Madaan, Nicholas Roberts, Nikolay Bashlykov, Ajay Menon, Vincent Moens, Amar Budhiraja, Despoina Magka, Vladislav Vorotilov, Gaurav Chaurasia, Dieuwke Hupkes, Ricardo Silveira Cabral, Tatiana Shavrina, Jakob Foerster, Yoram Bachrach, William Yang Wang, and Roberta Raileanu. Mlgym: A new framework and benchmark for advancing ai research agents. *arXiv preprint arXiv:2502.14499*, 2025. URL https://arxiv.org/abs/2502.14499.

Mark W. Nichols. The impact of visiting team travel on game outcome and biases in nfl betting markets. *Journal of Sports Economics*, 15(1):78–96, 2014. doi: 10.1177/1527002512440580.

Alexander Novikov, Ngân Vũ, Marvin Eisenberger, Emilien Dupont, Po-Sen Huang, Adam Zsolt Wagner, Sergey Shirobokov, Borislav Kozlovskii, Francisco J. R. Ruiz, Abbas Mehrabian, M. Pawan Kumar, Abigail See, Swarat Chaudhuri, George Holland, Alex Davies, Sebastian Nowozin, Pushmeet Kohli, and Matej Balog. Alphaevolve: A coding agent for scientific and algorithmic discovery, 2025. URL https://arxiv.org/abs/2506.13131.

Open Reward Standard. Open reward standard. https://openrewardstandard.io, 2026. Documentation website. Accessed 2026-04-05.

Premier League. The numbers that show this has been a season like no other, May 2024. URL https://www.premierleague.com/en/news/4031506.

Frank P. Ramsey. Truth and probability. In R. B. Braithwaite, editor, *The Foundations of Mathematics and Other Logical Essays*, pages 156–198. Kegan, Paul, Trench, Trubner & Co., London, 1931. Essay written in 1926.

Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and Maksym Andriushchenko. Posttrainbench: Can llm agents automate llm post-training? *arXiv preprint arXiv:2603.08640*, 2026. URL https://arxiv.org/abs/2603.08640.

Claude E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3–4): 379–423, 623–656, 1948. doi: 10.1002/j.1538-7305.1948.tb01338.x.

Ross Taylor. A tour of time series analysis (and a model for predicting nfl games). GitHub repository / presentation materials, 2016. URL https://github.com/RJT1990/PyData2016-SanFrancisco. Referenced by the PyFlux GASRank documentation; accessed 2026-04-04.

Edward O. Thorp. The kelly criterion in blackjack, sports betting, and the stock market. In *Proceedings of the 10th International Conference on Gambling and Risk Taking*, Montreal, June 1997. URL https://sites.oxy.edu/lengyel/M330/thorp/paper1.pdf. Revised May 29, 1998.

Edward O. Thorp. A perspective on quantitative finance: Models for beating the market. In Paul Wilmott, editor, *The Best of Wilmott 1: Incorporating the Quantitative Finance Review*, pages 33–38. John Wiley & Sons, Chichester, 2005.

Edward O. Thorp. The kelly criterion in blackjack, sports betting, and the stock market. In Stavros A. Zenios and William T. Ziemba, editors, *Handbook of Asset and Liability Management*, volume 1, pages 385–428. Elsevier, Amsterdam, 2008. doi: 10.1016/B978-044453248-0.50015-0.

Edward O. Thorp and Sheen T. Kassouf. *Beat the Market: A Scientific Stock Market System*. Random House, New York, 1967.

Nils Van Damme and Stijn Baert. Home advantage in european international soccer: Which dimension of distance matters? *Economics*, 13(1):20190050, 2019. doi: 10.5018/economics-ejournal.ja.2019-50.

Keyi Wang, Nikolaus Holzer, Ziyi Xia, Yupeng Cao, Jiechao Gao, Anwar Walid, Kairong Xiao, and Xiao-Yang Liu Yanglet. FinRL contests: Benchmarking data-driven financial reinforcement learning agents. *arXiv preprint arXiv:2504.02281*, 2025. URL https://arxiv.org/abs/2504.02281.

Nicholas Watanabe, Pamela Wicker, and Grace Yan. Weather conditions, travel distance, rest, and running performance: The 2014 fifa world cup and implications for the future. *Journal of Sport Management*, 31(1):27–43, 2017. doi: 10.1123/jsm.2016-0077.

Jack Wildman, Nikos I. Bosse, Daniel Hnyk, Peter Mühlbacher, Finn Hambly, Jon Evans, Dan Schwarz, and Lawrence Phillips. Bench to the future: A pastcasting benchmark for forecasting agents. *arXiv preprint arXiv:2506.21558*, 2025. doi: 10.48550/arXiv.2506.21558.

Xixi Wu, Kuan Li, Yida Zhao, Liwen Zhang, Litu Ou, Huifeng Yin, Zhongwang Zhang, Xinmiao Yu, Dingchu Zhang, Yong Jiang, Pengjun Xie, Fei Huang, Minhao Cheng, Shuai Wang, Hong Cheng, and Jingren Zhou. Resum: Unlocking long-horizon search intelligence via context summarization, 2025. URL https://arxiv.org/abs/2509.13313.

Qingchuan Yang, Simon Mahns, Sida Li, Anri Gu, Jibang Wu, and Haifeng Xu. Llm-as-a-prophet: Understanding predictive intelligence with prophet arena. In *International Conference on Learning Representations*, 2026. URL https://openreview.net/forum?id=VpiHkMSPqI.

Xiaochuang Yuan, Hui Xu, Silvia Xu, Cui Zou, and Jing Xiong. TraderBench: How robust are AI agents in adversarial capital markets? *arXiv preprint arXiv:2603.00285*, 2026. URL https://arxiv.org/abs/2603.00285.

Zhiyuan Zeng, Jiashuo Liu, Siyuan Chen, Tianci He, Yali Liao, Yixiao Tian, Jinpeng Wang, Zaiyuan Wang, Yang Yang, Lingyue Yin, et al. Futurex: An advanced live benchmark for llm agents in future prediction. *arXiv preprint arXiv:2508.11987*, 2025. doi: 10.48550/arXiv.2508.11987.

Model	ML Approach	Staking (executed)	Adapt.	Characteristic Failure
Claude Opus 4.6	Calibrated GBM / RF+LR ensembles; one seed pivoted to Poisson+market blend	Frac. Kelly 0.15–0.25	Partial	Draw/under-2.5 miscalibration, low sophistication in dealing with non-stationary factors
GPT-5.4	Walk-forward LR / dual-LR ensemble with Elo and rolling form	Rule-based %; pivoted to penny bets	Full	Front-loaded modelling; conceded edge, reverted to capital preservation
Gemini 3.1 Pro	RF on bookmaker odds (base); XGBoost with daily retrain (best seed)	Flat or frac-Kelly 0.05–0.10	Partial	Locked strategy on matchday 2; very high seed variance (bankruptcy vs. +33.7%)
Gemini Flash 3.1	RF on team-ID or all-time win rates	Ad hoc round-number bets	None	Zero-sample teams treated as zero-probability; oversized single wagers; 2/3 seeds forfeited.
GLM-5	Bradley-Terry + Poisson; later RF+GB ensemble	Flat / quarter-Kelly (floored)	None	Hardcoded 25% draw rate; self-critiques never implemented
Kimi K2.5	GBM with rolling form; one seed inline Poisson xG	Kelly coded but never invoked	None	Tool-call formatting failures; 98%-of-bankroll accidental bet; 2/3 seeds forfeited
Arcee Trinity	RF+Poisson scaffold; 17-feature RF	Coded frac-Kelly, rarely executed	None	2/3 seeds placed zero bets; inference-time feature collapse
Grok 4.20	W/D/L strength ratio; team-ID RF; xG LR (label leak)	None; flat or escalating round bets	None	Label leakage in its backtesting scripts; terminal "task complete" loops mid-season

Table 6 | **Per-Model Strategy Summary.** Aggregated across three seeds per model. "Staking" reports what agents actually executed, not what they discussed in reasoning.

A. Per-model Narratives

Here we provide detailed seed-level narratives for each of the eight models evaluated in the betting benchmark. These supplement the failure-mode analysis in the main text with model-specific context on strategy, tool usage, and trajectory.

A.1. Claude Opus 4.6 (−11.0% ROI, £89,035).

Opus was the only model to avoid ruin across all three seeds. It consistently deployed calibrated gradient boosting ensembles with fractional Kelly staking, and showed meaningful adaptation. Its best seed (−0.2% ROI) pivoted from a gradient boosting and Random Forest ensemble to a Poisson-plus-market-prior blend after diagnosing early losses, then identified profitable patterns in away favorites at short odds and high expected-goal over 2.5 markets, while its weakest seed (−18.8%) was undermined by placing unprofitable bets on matchdays where no model edge was identified. Across seeds, Opus demonstrated the most disciplined bankroll management in the benchmark. It also showed a recurring tension around promoted teams: it correctly diagnosed model errors on fixtures involving Luton, Burnley, and Sheffield United but repeatedly deferred to its model anyway, reasoning that it should follow the model-based approach since that was the framework it was using. One seed reduced its Kelly fraction from 0.25 to 0.15 after drawdowns, and another pivoted from gradient boosting to a Poisson-plus-market-prior blend after diagnosing that its model had no edge against the closing line.

A.2. GPT-5.4 (−13.6% ROI, £86,365).

The second-best model on average and the most engineering-intensive. One seed devoted roughly 160 tool calls to model building before placing its first bet, constructing a walk-forward validated logistic regression pipeline with Elo ratings and rolling form features. After determining that its model could not reliably outperform the market (model log-loss 0.974 vs. market 0.971), it progressively reduced stakes to £0.01 penny bets for the majority of the season, preserving capital. Seed variance was substantial: one seed lost 31.6% despite comparable strategy sophistication (13/44 vs. 18/44 win rate), while another achieved near break-even (−4.1%) with a dual logistic regression ensemble and four-tier fallback hierarchy. Across seeds, GPT-5.4 discussed Kelly staking extensively in its reasoning chains but implemented simpler percentage-of-bankroll rules in practice. The best-performing seed adopted the most elegant approach to promoted teams, relying on `OneHotEncoder(handle_unknown='ignore')` so that bookmaker odds absorbed the promotion signal, in effect deferring to the market on teams it could not model.

A.3. Gemini 3.1 Pro Preview (−43.3% ROI, £56,715).

This model exhibited the widest variance of any competitor: one seed went bankrupt (£0), another achieved the single best ROI in the entire benchmark (+33.7%), and the third bled steadily to a 63.6% loss. The profitable seed used two XGBoost classifiers with daily retraining and disciplined fractional Kelly sizing. This was the only Gemini variant to retrain its model, though a critical flaw in its staking code (using a fixed £200 pseudo-bankroll rather than the actual bankroll) prevented compounding. The worst-performing seed trained a Random Forest solely on bookmaker odds as features, producing predictions that added noise rather than signal. After a fortunate early £100 flat bet on Man City, it locked its strategy on matchday 2 and never adapted across the remaining 36 matchdays.

A.4. Gemini Flash 3.1 (−58.4% ROI, £41,605).

Two of three seeds forfeited mid-season with near-zero bankrolls. One seed reached a peak of £1,219.60 (5.5× starting bankroll) by backing strong favourites, but with no bankroll discipline it subsequently wagered £600 (63% of its bankroll) on a single match and lost everything. The only profitable seed (+24.7%) relied on a Random Forest trained on label-encoded team identities as its only features, running on flat £10 stakes with zero visible reasoning. No seed used Kelly staking or any systematic staking formula. Flash also treated Luton's historical win rate of zero as a genuine signal rather than missing data, leading to repeated bets against Luton that contributed to its bankruptcy.

A.5. GLM-5 (−58.8% ROI, £41,221).

One of three seeds went bankrupt. One seed used a hand-built Bradley-Terry model with Poisson goal predictions and achieved a 55.2% win rate but still lost 14.3%, recovering from a low of £93.30 by pivoting mid-season from value betting to conservative favourite-backing. Across all three seeds, GLM-5 displayed a distinctive pattern of diagnosis without action: it wrote multiple self-critique documents correctly identifying its own calibration failures, including a hardcoded 25% draw rate and overestimation of home advantage, but never translated these insights into model corrections. Another seed used a more sophisticated Random Forest plus gradient boosting ensemble with quarter-Kelly staking, but the environment's £1 minimum bet floor converted fractional Kelly into effectively flat staking as the bankroll dwindled, and it eventually went bankrupt.

A.6. Kimi K2.5 (−92.6% ROI, £7,420).

One seed went bankrupt. It built a well-designed GradientBoosting pipeline with 18 rolling form features and fractional Kelly staking code, but the prediction script was never actually executed. A persistent tool-call formatting bug caused the model to send `": ["` as a bash command approximately 50 times in sequence. When the model resorted to manual flat-stake betting, a single malformed tool call placed a £251 wager (98% of the remaining bankroll) on Burnley versus Luton, causing immediate bankruptcy. Another seed achieved a 62.3% win rate (the highest of any seed) using inline Poisson expected-goals modelling, but abandoned this approach mid-season in favour of a favourite-betting strategy, declining from a peak of £373 to £160 before forfeiting mid-season. The bankrupt seed also declared the task "complete" six times mid-season while its bankroll continued to fall, and entered an 18-matchday plateau where no bets were placed.

A.7. Grok 4.20 (−100.0% ROI, £0).

One seed went bankrupt by matchday 15 with a 9.5% win rate. Another built a Random Forest using only label-encoded team names as features, producing absurd predictions such as newly promoted Burnley having a 76.6% chance of beating Man City, and quit after 14 matchdays. The third constructed a logistic regression on post-match expected goals data, a form of label leakage that inflated in-sample accuracy to 67%, which was then celebrated as a genuine improvement. Two of three seeds declared "task complete" prematurely while the season was still running; the third went bankrupt by matchday 15.

A.8. Arcee Trinity (−100.0% ROI, £0).

The worst performing model across all seeds. Two of three seeds placed zero bets, failing to understand or call the provided tools entirely. One seed wrote a plausible Random Forest and Poisson scaffold but attempted to call environment functions via Python `subprocess` rather than the provided tools, silently failing throughout. Another became stuck in a preprocessing debug loop and terminated with a `NameError` mid-rewrite. The third spent two thirds of its trajectory writing and rewriting Python scripts rather than interacting with the betting environment. When the model finally placed bets in the remaining seed, it escalated from £5–10 to £100 stakes without systematic justification, degraded into a repetitive loop, and forfeited mid-season with a bankroll of £15,773. At inference time, its Random Forest received post-match goal counts and the realised result as input features, all of which were set to zero when querying for predictions, effectively collapsing the model to team identity alone.

B. Example Matchday Loop: Claude Opus 4.6, Matchday 1

To give a concrete sense of the agent–environment interaction, we reproduce the full tool-call loop for the opening matchday of Claude Opus 4.6's best seed (−0.2% ROI). The loop has five stages: observe fixtures, run the trained model, reason about value, place bets, and settle. Earlier in the episode the agent had already trained a gradient-boosting and Random Forest ensemble over ten seasons of historical data (`model.py`) and written a betting-strategy script (`bet_today.py`) that applies the model to the current matchday and reports any value bets found. The loop below begins with the single match on 11/08/2023.

view_matches()

```
Matches for 11/08/2023:

Match 0: Burnley vs Man City
  Odds - Home: 8.0, Draw: 5.5, Away: 1.33
  Over 2.5: 1.67, Under 2.5: 2.2
```

bash(command="python3 bet_today.py")

```
Current Bankroll: £220.00
Strategy: ML Value Betting with Quarter-Kelly Sizing

Analyzing: Burnley vs Man City (Match 0)

Model Predictions:
  Home win: 40.1% (odds: 8.0, implied: 12.5%)
  Draw:     26.9% (odds: 5.5, implied: 18.2%)
  Away win: 33.0% (odds: 1.33, implied: 75.2%)

  VALUE BETS FOUND:
    HOME: edge=220.7%, model_prob=40.1%, kelly=7.9%, bet=£17
    DRAW: edge=48.2%, model_prob=26.9%, kelly=2.7%, bet=£6
    UNDER_2_5: edge=8.9%, model_prob=49.5%, kelly=1.9%, bet=£5

  Total exposure: £28 (12.7% of bankroll)
```

Assistant reasoning

```
The model has identified value bets. Now let me place them based on the ML model's
recommendations.
```

place_bet(match_id=0, bet_type="home", amount=17)

```
Bet placed: 17.0 on home for Burnley vs Man City at odds 8.0.
Potential return: 136.00
```

place_bet(match_id=0, bet_type="draw", amount=6)

```
Bet placed: 6.0 on draw for Burnley vs Man City at odds 5.5.
Potential return: 33.00
```

place_bet(match_id=0, bet_type="under_2_5", amount=5)

```
Bet placed: 5.0 on under\_2\_5 for Burnley vs Man City at odds 2.2.
Potential return: 11.00
```

next_matchday()

```
Bet Results:

x LOST: Burnley vs Man City (home) - Lost: £17.00
x LOST: Burnley vs Man City (draw) - Lost: £6.00
x LOST: Burnley vs Man City (under_2_5) - Lost: £5.00

Net result: £-28.00
New bankroll: £192.00

Advanced to next matchday: 12/08/2023
```

The opening matchday illustrates a characteristic early-season failure: the model assigned 40.1% to a Burnley home win against an implied 12.5%, claiming a 220% edge on a newly promoted side facing

the reigning champions.

C. Long-Horizon Reasoning Patterns

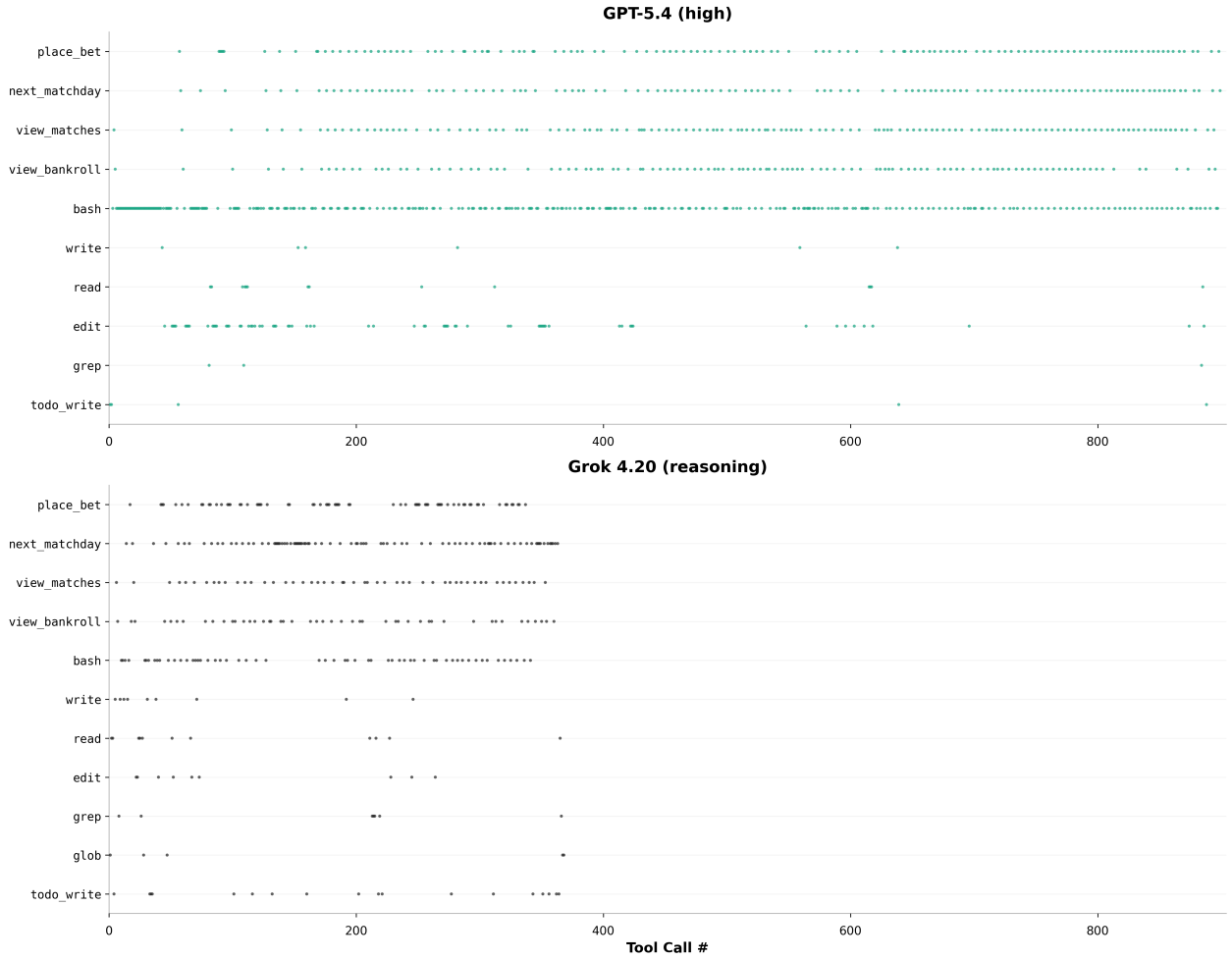


Figure 7 | **Tool Use Plot of GPT 5.4 vs Grok 4.20**. GPT 5.4 plans for longer, developing its model using many bash tool calls at the start of the episode before placing bets and advancing the matchday, whereas Grok 4.20 is more impatient. In mid and late season, GPT 5.4 consistently uses bash to execute and refine its strategies, whereas Grok tends to stick to a single strategy with less refinement.

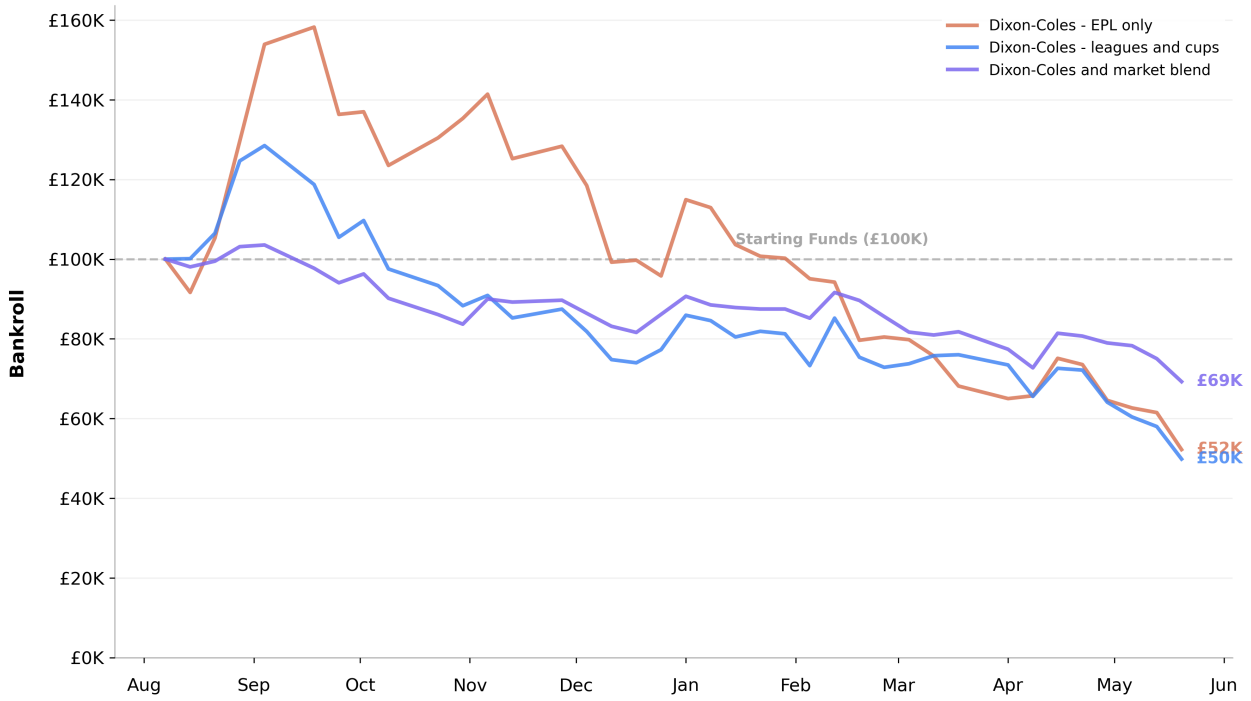


Figure 8 | **Basic Football Modelling Baselines**. Dixon-Coles type models are a seminal yet outdated type of baseine for modelling football outcomes. Despite this, the best performing baseline for this type of model outperforms many of the frontier models we evaluated in the main body of the paper.

D. Classic Football Modeling Baselines

To see how classic football modeling baselines perform, we backtested (a) a Dixon-Coles model with Premier League data only, (b) a Dixon-Coles model with multiple league and cup data, and (c) a Dixon-Coles model blended with market probabilities (Dixon and Coles, 1997). We used fractional Kelly ($f = 0.25f^*$) for each. Strategy (a) and (b) have sophistication scores of $7/44 = 16\%$. Strategy (c) has a sophistication scores of $11/44 = 25\%$.

The best performing baseline (c) achieves a final bankroll of 69k, which is above all models other than Opus 4.6 and GPT-5.4. Dixon-Coles is an outdated 2000s baseline which doesn't utilise all available data or account for non-stationarity in a principled way. It is therefore even more surprising that many frontier models, such as Gemini 3.1 Pro, are unable to beat or match it on `KellyBench`.

E. Tool Specifications

Here we document the tools available to agents in the betting environment. Agents interact with a simulated Premier League season by viewing fixtures, placing bets, managing bankroll, and advancing through matchdays. In addition, a CLI toolset mirroring the Claude Code interface is available for model development within a sandboxed compute environment.

E.1. `view_matches`

Purpose. Display the fixtures and bookmaker odds for the current matchday.

Parameters. None.

Returns. A formatted list of matches for the current date. Each entry includes the match ID, home and away team names, and decimal odds for the home/draw/away markets (sourced from Bet365, Gamebookers, or Interwetten in fallback order). Where available, over/under 2.5 goals odds are also displayed.

E.2. `place_bet`

Purpose. Place a wager on a specific match outcome.

Parameters.

- `match_id` (int): Index of the match on the current matchday, as returned by `view_matches`.
- `bet_type` (str): One of `"home"`, `"draw"`, `"away"`, `"over_2_5"`, or `"under_2_5"`.
- `amount` (float): Stake in pounds. Must be positive and at most the current bankroll.

Returns. Confirmation of the placed bet, including the matched odds and potential return. The stake is immediately deducted from the bankroll. Multiple bets may be placed per matchday (including on the same match).

E.3. `view_bankroll`

Purpose. Query the agent's current financial state.

Parameters. None.

Returns. The current bankroll balance. If bets have been placed on the current matchday, also reports the total value staked and the amount still available for betting.

E.4. `next_matchday`

Purpose. Settle all pending bets and advance the simulation to the next set of fixtures.

Parameters. None.

Precondition. At least one bet must have been placed on the current matchday; the tool refuses to advance otherwise.

Returns. A settlement report listing each bet as won or lost, with stakes and payouts. Winning bets pay out at the quoted decimal odds (stake $\times$ odds). The report includes the net result, the updated bankroll, and the date of the next matchday. Updated match data (results, player statistics) is downloaded to `/home/ubuntu/latest_data/` for use in model retraining. When the final matchday is settled, the environment terminates and reports the final bankroll and total profit/loss.

Reward signal. The per-step reward is the log wealth ratio $\ln(B_{t+1}/B_t)$, where B_t is the bankroll before settlement and B_{t+1} is the bankroll after. This incentivises Kelly-optimal growth.

E.5. `bash`

Purpose. Execute arbitrary shell commands in a sandboxed Linux environment for data analysis and model development.

Parameters.

- `command` (str): The shell command to execute.
- `description` (str, optional): A brief note on the command's intent.
- `timeout` (float, optional): Maximum execution time in seconds (default 30).

Returns. Standard output and the exit code. Network access is blocked: commands matching common HTTP/socket patterns (e.g. `curl`, `requests`, `urllib`) are rejected, and three such attempts result in disqualification with bankroll elimination. Historical match data is pre-loaded at `/tmp/gr-datasets/`.

E.6. `read`

Purpose. Read the contents of a file in the sandbox.

Parameters.

- `file_path` (str): Path to the file to read.
- `offset` (int, optional): Starting line number.
- `limit` (int, optional): Maximum number of lines to return.

Returns. Numbered file contents (or the requested slice). When both `offset` and `limit` are provided, returns lines in the range [`offset`, `offset` + `limit`].

E.7. `write`

Purpose. Create or overwrite a file with the given content.

Parameters.

- `file_path` (str): Destination path. Parent directories are created automatically.
- `content` (str): The full file content to write.

Returns. Confirmation of successful write.

E.8. `edit`

Purpose. Perform exact string replacement within an existing file.

Parameters.

- `file_path` (str): Path to the file to edit.
- `old_string` (str): The substring to find. Must appear exactly once unless `replace_all` is set.
- `new_string` (str): The replacement string.
- `replace_all` (bool, optional): If true, replaces all occurrences. Default false.

Returns. Confirmation of successful edit. Fails if `old_string` is not found or is ambiguous (appears

more than once with `replace_all=false`).

E.9. `glob`

Purpose. Find files matching a glob pattern.

Parameters.

- `pattern` (str): Filename glob pattern (e.g. `"*.csv"`).
- `path` (str, optional): Root directory to search from. Defaults to the working directory.

Returns. A sorted list of matching file paths.

E.10. `grep`

Purpose. Search for a regex pattern across files.

Parameters.

- `pattern` (str): Regular expression to match.
- `path` (str, optional): Root directory. Defaults to the working directory.
- `glob` (str, optional): If provided, restricts the search to files matching this glob pattern.

Returns. Matching lines with file paths and line numbers.

E.11. `todo_write`

Purpose. Manage a structured to-do list for task planning and progress tracking.

Parameters.

- `todos` (list[dict]): A list of to-do items, each with fields `id`, `content` (str), `status` (one of `"pending"`, `"in_progress"`, `"completed"`), and `priority` (one of `"high"`, `"medium"`, `"low"`).

Returns. A formatted rendering of the current to-do list. The list is replaced in full on each call (i.e. last-write-wins).